Web APIs from the Ground Up

How the web talks: HTTP, JSON and REST from zero. You'll build a working API by hand in plain Python, then see the same thing in Flask and Django. Read this first, then Guide B (the Django guide).

How to use this guide

- Read it in order. Each section builds on the one before, and nothing is assumed except basic Python.
- **Try it** boxes are things to type and run on your Mac. Type the code instead of copying it: typing is where it sticks.
- **Check yourself** boxes are quick questions. Answer them in your head, then check section 13.
- **Practice** boxes point to exercises in the **api-drills** repo (this is Guide A there; see its README and PLAN.md). Exercises ex01 and ex02 belong to this guide.
- Plan on about 3 to 4 hours of reading and trying, plus the two exercises. Every code example was run before going into this guide.

0. What you need

- **Terminal** (Applications > Utilities > Terminal). The prompt looks like `neil-m1 ~ %`.
- **Python 3**: check with `python3 --version`.
- **curl**, a command-line tool that sends web requests. It's already on every Mac: check with `curl --version`.
- **A code editor** such as VS Code.
- A scratch folder for the Try it code: `mkdir ~/api-basics && cd ~/api-basics`. (The exercises live in the api-drills repo; its README covers setup.)

1. The big picture: clients, servers and APIs

Almost everything on the web is two programs exchanging messages:

- A **client** sends a **request**: "give me the list of todos" or "save this new todo". Your browser is a client; so is a phone app, curl, or a test.
- A **server** is a program that waits for requests and sends back a **response**: the data, plus a short code saying how it went.

A **web page** response is HTML, meant for humans to look at. An **API** (Application Programming Interface) response is data, usually JSON, meant for other programs to use. When a weather app shows the forecast, it sent a request to a weather API and drew the JSON it got back.

A useful picture is a restaurant. The **menu** is the API's documentation: the list of things you're allowed to ask for and how to ask. Your **order** is the request. The **kitchen** is the server, which you never see inside. The **dish** is the response. If you order something not on the menu, you get a polite "we don't do that" (an error code) instead of a dish.

In the Goldman exam, **you** build the kitchen. The client is a set of automated tests that place orders and check that each dish is exactly what the menu promised.

2. Addresses: URLs

Every request is sent to a URL. Take this one apart:

```
http://127.0.0.1:8000/todos/3?done=true&sort=title
```

Part	Here	Meaning
scheme	`http`	the protocol (the language) to speak; `https` is the encrypted version
host	`127.0.0.1`	which computer. `127.0.0.1` (also called `localhost`) always means **this** computer
port	`8000`	which program on that computer. Each server listens on its own port number
path	`/todos/3`	which thing you want on that server: here, todo number 3
query string	`?done=true&sort=title`	optional extras, as `key=value` pairs joined by `&`

Websites you know usually hide the port: `https` uses 443 by default. When you develop on your own machine you'll use addresses like `http://127.0.0.1:8000`. Special characters in a URL are encoded, e.g. a space becomes `%20`.

Check yourself (answers in section 13)

1. In `http://127.0.0.1:8000/todos/3?done=true`, what are the port, the path and the query string?

3. HTTP: the conversation

HTTP is the format of the messages. It's plain text, and you'll see it for real in section 5. A **request** has four parts:

```
POST /todos HTTP/1.1                <- method, path, protocol version
Host: 127.0.0.1:8000                <- headers: one "Name: value" per line
Content-Type: application/json
Content-Length: 21
                                    <- a blank line ends the headers
{"title": "Buy milk"}               <- the body (optional)
```

And a **response**:

```
HTTP/1.1 201 Created                <- status code and its name
Content-Type: application/json      <- headers

{"id": 1, "title": "Buy milk"}      <- the body
```

Methods: the verb of the request

Method	Means	Has a body?
`GET`	read something. Never changes anything on the server	no
`POST`	create something new (or trigger an action)	yes
`PUT`	replace something completely	yes
`PATCH`	change part of something	yes
`DELETE`	delete something	usually no

Two words you'll hear: a **safe** method doesn't change anything (GET). An **idempotent** method has the same effect whether you send it once or five times (GET, PUT, DELETE: deleting todo 3 twice still leaves it deleted). POST is neither: sending it twice creates two todos.

Status codes: how did it go?

The first digit tells you the family: **2xx** success, **3xx** go somewhere else, **4xx** the **client** made a mistake, **5xx** the **server** broke.

Code	Name	When you use it
200	OK	a normal successful response with a body
201	Created	a POST successfully created something; the body is the new thing
204	No Content	success, nothing to send back (typical after DELETE)
301	Moved Permanently	"this lives at another URL" (e.g. the same URL with a slash)
400	Bad Request	the request is invalid: bad JSON, a missing field, a negative price
401 / 403	Unauthorized / Forbidden	not logged in / logged in but not allowed
404	Not Found	no such thing at that path (e.g. todo 99 doesn't exist)
405	Method Not Allowed	the path exists but doesn't accept that method
409	Conflict	valid request, but it clashes with the current state
500	Internal Server Error	the server crashed while handling it: a bug
501	Not Implemented	the server doesn't support that method at all

Headers you'll meet

- `Content-Type: application/json`: "the body is JSON". Servers and clients use it to decide how to read the body.
- `Content-Length: 21`: the body is 21 bytes long, so the reader knows where it ends.
- `Accept` (what the client wants back) and `Authorization` (a password or token).

Finally, HTTP is **stateless**: every request stands alone and carries everything needed to answer it. The server doesn't remember your previous request unless it stored something in a database.

Check yourself (answers in section 13)

1. Which status code fits each case? (a) a todo was created; (b) todo 99 doesn't exist; (c) DELETE sent to a URL that only allows GET; (d) the server code raised an exception; (e) the body was missing the title.
2. What's the difference between PUT and PATCH?

4. JSON: the data format

JSON (JavaScript Object Notation) is text that represents data. It looks almost like Python:

```
{
  "id": 1,
  "title": "Buy milk",
  "done": false,
  "tags": ["shopping", "home"],
  "dueDate": null
}
```

JSON	Python	JSON	Python
object `{...}`	`dict`	`true` / `false`	`True` / `False`
array `[...]`	`list`	`null`	`None`
string `"text"`	`str`	number `42`, `4.5`	`int`, `float`

JSON is **strict**: strings and keys use double quotes only, no trailing commas, no comments, and it's `true` not `True`. Python converts with the `json` module:

```
import json

text = '{"title": "Buy milk", "done": false}'
data = json.loads(text)          # text -> Python:  {'title': 'Buy milk', 'done': False}
data["done"] = True
back = json.dumps(data)          # Python -> text:  '{"title": "Buy milk", "done": true}'
```

Turning Python data into JSON text is called **serializing**; the reverse is **deserializing** or **parsing**. Remember: what travels over HTTP is always text (bytes). `json.loads` is the step that turns it into a dict you can use.

Try it
Start `python3` and run the four lines above. Then try `json.loads("{'title': 'x'}")` and read the error: that's what a server sees when a client sends bad JSON. Type `exit()` to leave.

Check yourself (answers in section 13)

1. What's wrong with `{'title': 'milk', 'done': True,}` as JSON? (Three things.)
2. What does `json.loads('[1, {"a": null}]')` return?

Practice in api-drills: ex01_json (30 min): take URLs apart, classify status codes, parse and validate JSON. Run `python score.py ex01`.

5. Your first server, by hand

A server is just a program that loops forever: wait for a request, work out a response, send it, repeat. Python's standard library can do this with no installs. Create `hello_server.py`:

```
import json
from http.server import BaseHTTPRequestHandler, HTTPServer


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        print("Request:", self.command, self.path)
        body = json.dumps({"message": "hello"}).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(body)


print("Listening on http://127.0.0.1:8000")
HTTPServer(("127.0.0.1", 8000), Handler).serve_forever()
```

- `HTTPServer(("127.0.0.1", 8000), Handler)` listens on port 8000 of this computer; `serve_forever()` is the loop.
- For every GET request, Python calls `do_GET`. `self.path` holds the path (`/hello`).
- The response is written in the same order as the raw HTTP in section 3: status (`send_response`), headers, a blank line (`end_headers`), then the body bytes.
- This server answers **every** GET path the same way: it doesn't look at the path yet.

Try it

1. Run `python3 hello_server.py`. It prints `Listening on http://127.0.0.1:8000` and waits.
2. Open **a second Terminal window** (Cmd+N) and run `curl http://127.0.0.1:8000/hello`. You get `{"message": "hello"}`.
3. Run `curl -i http://127.0.0.1:8000/hello`. The `-i` shows the status line and headers: `HTTP/1.0 200 OK`, `Content-Type: application/json`...
4. Open the same URL in your browser. Look at the first window: the server printed a line for each request.
5. Stop the server with **Ctrl+C**. If you ever see "Address already in use", an old server is still running on that port: find and stop it, or use another port.

6. curl cheat sheet

curl is how you'll poke at any API, including your Django ones later.

```
curl http://127.0.0.1:8000/todos                    # GET, print the body
curl -i http://127.0.0.1:8000/todos                 # also show status line and headers
curl -v http://127.0.0.1:8000/todos                 # show everything sent and received

curl -X POST http://127.0.0.1:8000/todos \
     -H "Content-Type: application/json" \
     -d '{"title": "Buy milk"}'                      # POST a JSON body

curl -X PATCH ...   curl -X DELETE ...              # other methods: -X plus the name

curl "http://127.0.0.1:8000/todos?done=true&sort=title"   # quote URLs that have ? or &

curl -s -o /dev/null -w "%{http_code}\n" URL          # print only the status code
```

Mac gotcha: the Mac's shell (zsh) treats ? and & as special characters. If curl complains "no matches found" or behaves oddly, put the URL in double quotes. The JSON body goes in single quotes so the double quotes inside it survive.

7. Build a tiny API by hand

Now a real (tiny) API: a todo list you can add to and read from. Create `todos_server.py`:

```
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

todos = []      # our "database": a plain list, lost when the server stops
nextId = 1


class Handler(BaseHTTPRequestHandler):
    def sendJson(self, status, data):
        body = json.dumps(data).encode()
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(body)
    def do_GET(self):
        if self.path == "/todos":
            self.sendJson(200, todos)
        elif self.path.startswith("/todos/"):
            todoId = self.path[len("/todos/"):]
            for todo in todos:
                if str(todo["id"]) == todoId:
                    self.sendJson(200, todo)
                    return
            self.sendJson(404, {"error": "no such todo"})
        else:
            self.sendJson(404, {"error": "unknown URL"})
    def do_POST(self):
        global nextId
        if self.path != "/todos":
            self.sendJson(404, {"error": "unknown URL"})
            return
        length = int(self.headers.get("Content-Length", 0))
        data = json.loads(self.rfile.read(length))
        todo = {"id": nextId, "title": data["title"]}
        nextId += 1
        todos.append(todo)
        self.sendJson(201, todo)


print("Listening on http://127.0.0.1:8000")
HTTPServer(("127.0.0.1", 8000), Handler).serve_forever()
```

What each piece does:

- `todos` is the "database": a Python list living in the server's memory. `nextId` hands out ids 1, 2, 3...
- `sendJson` is a helper so every response is built the same way.
- `do_GET` does its own **routing**: it reads `self.path` and decides which code to run. `/todos` returns the whole list (200); `/todos/<id>` searches for one (200) or answers 404.
- `do_POST` reads exactly `Content-Length` bytes of body, parses the JSON, creates the todo, and answers **201 Created** with the new todo, including its id.

Try it

1. Run `python3 todos_server.py` in one window. In the other window:
2. `curl -X POST http://127.0.0.1:8000/todos -H "Content-Type: application/json" -d '{"title": "Buy milk"}'` gives `{"id": 1, "title": "Buy milk"}`. POST a second one.
3. `curl http://127.0.0.1:8000/todos` gives the list of both. `curl -i http://127.0.0.1:8000/todos/2` gives 200; `/todos/9` gives 404.

Three experiments that show what's missing

Do this	What happens	What it teaches
`curl -i -X DELETE http://127.0.0.1:8000/todos/1`	`501 Unsupported method ('DELETE')`: there's no `do_DELETE`	Every method needs code, and unsupported ones need a proper answer (405)
POST broken JSON: `-d '{"title": "oops"'`	curl gets no response; the server window prints a `JSONDecodeError` traceback	Without error handling, one bad request crashes the handling of that request. It should be a 400
Stop the server (Ctrl+C), start it again, GET `/todos`	`[]`: everything is gone	Data in memory dies with the program. Real APIs store data in a database

Practice in api-drills: ex02_http_server (60-75 min): finish this server into a full todo API, with PATCH, DELETE, a `?done=true` filter, 400s for bad input, 404s and a 405. The tests start your server on a free port and send it real HTTP requests, just like curl. Run `python score.py ex02`.

By the end of ex02 you'll have felt every chore a framework does for you: matching paths, pulling ids out of URLs, reading bodies, turning crashes into error codes. That's the point of the next sections.

8. REST: the conventions

REST (REpresentational State Transfer) is not a library. It's a set of conventions for designing an API so that anyone can guess how it works. The core ideas:

- **Resources are nouns** at URLs: `/todos`, `/students`, `/products`. Use plural nouns, and no verbs in the path (`/createTodo` is not REST: the method is the verb).
- **Two URLs per resource:** the **collection** `/todos` and an **item** `/todos/3`.
- **The method says what to do**, and the status code says how it went.
- **Options go in the query string:** filtering, sorting, paging: `/todos?done=true&sort=title`.
- **Related things nest:** `/users/3/todos` means the todos belonging to user 3.
- **Stateless:** each request carries everything needed to answer it.

Request	Action (CRUD)	Success	Typical errors
`POST /todos`	Create one, from the JSON body	201 + the new todo	400
`GET /todos`	Read all (Read)	200 + array	-
`GET /todos/3`	Read one	200 + the todo	404
`PATCH /todos/3`	Update some fields	200 + updated todo	400, 404
`PUT /todos/3`	Replace entirely	200	400, 404
`DELETE /todos/3`	Delete	204, empty body	404
unsupported method	-	-	405

CRUD stands for Create, Read, Update, Delete: the four things almost every API does to its data. The Goldman sample question you saw was exactly this table for a `/students` resource.

Check yourself (answers in section 13)

1. Design the URLs and methods for a library API that can: (a) list all books, (b) add a book, (c) get book 7, (d) change book 7's title, (e) delete book 7, (f) list only books by Orwell, (g) list the loans of member 3, (h) mark loan 12 as returned.

9. Databases in one page

Your hand-made server lost everything on restart. A **database** stores data on disk and lets you search it quickly.

- A **relational database** holds **tables**. A table is like a spreadsheet: **columns** are fields (`id`, `title`, `done`), **rows** are records (one todo each).
- The **primary key** (usually `id`) uniquely identifies a row. The database hands out 1, 2, 3... automatically.
- You talk to it in **SQL**: `SELECT * FROM todo WHERE done = 1 ORDER BY id;` or `INSERT INTO todo (title) VALUES ('Buy milk');`
- **SQLite** is a database that lives in a single file. It's what Django uses by default and what the Goldman sample project used.
- An **ORM** (Object-Relational Mapper) lets you skip SQL: you write `Todo.objects.filter(done=True).order_by("id")` in Python and the ORM writes the SQL for you.
- A **migration** is a file describing a change to the tables ("add a column `done`"). When you change your Python model, you create a migration so the database's tables match.

10. Frameworks: Flask and Django

A **web framework** is a library that does the chores from section 7 for you, so you only write what's specific to your API:

Chore	By hand (section 7)	Framework
Routing (which code for which URL)	if/elif on `self.path`, slicing out the id	a route table; the id arrives as an argument
Reading the body	read `Content-Length` bytes, `json.loads`	one call
Building responses	status, headers, bytes by hand	one helper: data + status code
Unsupported methods	501 from the library	405 automatically
Crashes	request dropped	a 500 response; a helpful error page while developing
Storing data	a list in memory	an ORM and a real database (Django)
Testing	curl by hand	a test client that fakes requests

	Flask	Django
Style	small ("micro"): you add pieces as you need them	"batteries included": ORM, migrations, admin site, auth, test runner
Good for	small services, learning, prototypes	full applications; what the Goldman Diagnostics project used
Routes	decorators: `@app.get("/todos")`	a list in `urls.py`
Database	bring your own (e.g. SQLAlchemy)	built-in ORM + migrations

(You'll also hear of **FastAPI**, a newer Python framework built specifically for APIs. The ideas are identical.)

The same API in Flask

Install with `pip install flask` (inside a virtual environment, as in the drills README). Save as `todos_flask.py`:

```
from flask import Flask, jsonify, request

app = Flask(__name__)
todos = []
nextId = 1


@app.get("/todos")
def listTodos():
    return jsonify(todos)


@app.post("/todos")
def createTodo():
```

```
    global nextId
    data = request.get_json()
    todo = {"id": nextId, "title": data["title"]}
    nextId += 1
    todos.append(todo)
    return jsonify(todo), 201


@app.get("/todos/<int:todoId>")
def getTodo(todoId):
    for todo in todos:
        if todo["id"] == todoId:
            return jsonify(todo)
    return jsonify({"error": "no such todo"}), 404
```

Run it with `flask --app todos_flask run --port 8000` and use the same curl commands as before. (Use port 8000: on Macs, port 5000, Flask's default, is often taken by AirPlay Receiver.) Compare it with section 7: the routing, body parsing and response building have disappeared, and DELETE now gets a proper 405.

The same API in Django

Django splits the work across files. The data becomes a **model** (a database table) instead of a list:

```
# todos/models.py: the table
from django.db import models

class Todo(models.Model):
    title = models.CharField(max_length=200)
```

```
# todos/views.py: the code that answers requests
import json

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt

from .models import Todo

@csrf_exempt
def todoCollection(request):
    if request.method == "POST":
        data = json.loads(request.body)
        todo = Todo.objects.create(title=data["title"])
        return JsonResponse({"id": todo.id, "title": todo.title}, status=201)
    if request.method == "GET":
        todos = [{"id": t.id, "title": t.title} for t in Todo.objects.order_by("id")]
        return JsonResponse(todos, safe=False)
    return JsonResponse({"error": "method not allowed"}, status=405)
```

```
# todos/urls.py: which URL goes to which view
from django.urls import path

from . import views

urlpatterns = [
    path("todos", views.todoCollection),
]
```

Line for line it's the same logic as your hand-made server: check the method, parse the body, create, answer 201; or read all and answer 200. The difference is that the todos now survive a restart, because they're rows

in SQLite. The Django guide picks up exactly here and explains every line.

11. Testing an API

So far you tested by hand with curl. An **automated test** does the same thing in code: send a request, then **assert** what the response should be. Frameworks include a **test client** that sends fake requests straight into your code, without a real network or a running server:

```
response = client.post("/todos", {"title": "Buy milk"}, content_type="application/json")
assert response.status_code == 201
assert response.json() == {"id": 1, "title": "Buy milk"}
```

That's precisely how the Goldman exam scores you: a file of tests like this runs against your code, and your score is how many pass. In the drills you'll run them with `python manage.py test` and see a score with `python score.py`. The Django guide's section 9 shows the full syntax.

12. Glossary

Term	Meaning
API	a set of requests a program agrees to answer, with documented inputs and outputs
client / server	the program asking / the program answering
endpoint	one URL + method the API supports, e.g. `GET /todos/<id>`
resource	a kind of thing the API manages (todos, students); collection = all of them, item = one
request / response	the message sent / the message sent back
method	the verb of a request: GET, POST, PUT, PATCH, DELETE
status code	the 3-digit result of a response: 200, 201, 404, ...
header	a `Name: value` line of metadata, e.g. `Content-Type`
body / payload	the data part of a request or response (often JSON)
query string	the `?key=value&...` part of a URL, for options like filters
JSON	a text format for data: objects, arrays, strings, numbers, true/false/null
serialize / parse	turn data into text / turn text into data
route	a rule mapping a URL pattern to the code that handles it
view / handler	the function that handles a request and returns a response
framework	a library that handles the plumbing (Flask, Django, FastAPI)
CRUD	Create, Read, Update, Delete
REST	conventions for APIs: nouns in URLs, methods as verbs, status codes for results
database / table / row	where data persists / one kind of record / one record
primary key	the column that uniquely identifies a row (usually `id`)
ORM	lets you query the database with Python objects instead of SQL
migration	a file describing a change to the database tables
localhost / port	this computer / which program on it (`127.0.0.1:8000`)
stateless	each request stands alone; the server doesn't remember earlier ones
idempotent	sending it twice has the same effect as once (GET, PUT, DELETE)
CSRF	a browser attack Django blocks by default; why API views get `@csrf_exempt`

13. Answers to Check yourself

1. Port `8000`; path `/todos/3`; query string `done=true`.
2. (a) 201 Created; (b) 404 Not Found; (c) 405 Method Not Allowed; (d) 500 Internal Server Error; (e) 400 Bad Request.
3. PUT replaces the whole object, so you send every field. PATCH changes only the fields you send.
4. Single quotes instead of double quotes; `True` instead of `true`; a trailing comma before `}`.
5. `[1, {'a': None}]`: a Python list containing an int and a dict, with `null` turned into `None`.

Library API design (question 6), one good answer:

Task	Request
(a) list all books	`GET /books`
(b) add a book	`POST /books` with the book as JSON; 201
(c) get book 7	`GET /books/7`; 404 if missing
(d) change book 7's title	`PATCH /books/7` with `{"title": "..."}`
(e) delete book 7	`DELETE /books/7`; 204
(f) books by Orwell	`GET /books?author=Orwell` (a filter, so a query string)
(g) loans of member 3	`GET /members/3/loans` (nested), or `GET /loans?member=3`
(h) mark loan 12 returned	`PATCH /loans/12` with `{"returned": true}`. You'll also see action-style URLs like `PATCH /loans/12/return` (ex07 in the drills uses one); both are common.

14. Where next

1. Finish **ex01** and **ex02** in api-drills. ex02 is the most important exercise in the whole plan: once you've built routing, parsing and status codes by hand, Django's version of each will feel obvious.
2. Read **Guide B** (`docs/2_django_guide.pdf`) from section 1. It assumes exactly what this guide taught.
3. Carry on with **ex03** onwards, following `PLAN.md`.